



Searching Algorithms

Linear Search, Binary Search & Variations

Java 8+

THIRAVUGAL



Session Structure

1. Why searching matters
2. Linear Search
3. Binary Search
4. Binary Search Variations
5. Time & space complexity
6. Real-world systems
7. Interview-level problems



What is Searching?

Searching means:

Finding a target element in a dataset.

Goal:

Minimize time complexity.



Linear Search

Check each element one by one.

Works on:

- Unsorted arrays
- Lists

Linear Search Code

```
static int linearSearch(int[] arr, int key) {  
    for (int i = 0; i < arr.length; i++) {  
        if (arr[i] == key)  
            return i;  
    }  
    return -1;  
}
```

Time: $O(n)$

Space: $O(1)$



When to Use Linear Search?

- Small datasets
- Unsorted data
- One-time search



Binary Search

Requires:

Sorted array.

Idea:

Divide and conquer.

Each step halves search space.

Binary Search Code

```
static int binarySearch(int[] arr, int key) {  
    int left = 0, right = arr.length - 1;  
  
    while (left <= right) {  
        int mid = left + (right - left) / 2;  
  
        if (arr[mid] == key) return mid;  
        if (arr[mid] < key) left = mid + 1;  
        else right = mid - 1;  
    }  
    return -1;  
}
```

Time: $O(\log n)$

Space: $O(1)$



Why mid Calculation Matters?

Avoid overflow:

Wrong:

```
int mid = (left + right) / 2;
```

Correct:

```
int mid = left + (right - left) / 2;
```

Binary Search Recursion Version

```
static int binarySearchRec(int[] arr, int left, int right, int key) {  
    if (left > right) return -1;  
  
    int mid = left + (right - left) / 2;  
  
    if (arr[mid] == key) return mid;  
    if (arr[mid] < key)  
        return binarySearchRec(arr, mid + 1, right, key);  
    else  
        return binarySearchRec(arr, left, mid - 1, key);  
}
```

Time: $O(\log n)$

Space: $O(\log n)$ (recursion stack)



Real World Use Case — Database Index

Sorted data:

Binary search used for fast lookup.

Used in:

- B-Trees
- Database indexing
- File systems



Real World Use Case — Search Engines

Sorted inverted index.

Binary search speeds up word lookup.

First Occurrence of Element

Modified binary search.

```
static int firstOccurrence(int[] arr, int key) {  
    int left = 0, right = arr.length - 1;  
    int result = -1;  
  
    while (left <= right) {  
        int mid = left + (right - left) / 2;  
  
        if (arr[mid] == key) {  
            result = mid;  
            right = mid - 1;  
        }  
    }  
}
```



```
    else if (arr[mid] < key) {  
        left = mid + 1;  
    } else {  
        right = mid - 1;  
    }  
}  
return result;  
}
```

Time: $O(\log n)$

Last Occurrence

```
static int lastOccurrence(int[] arr, int key) {  
    int left = 0, right = arr.length - 1;  
    int result = -1;  
  
    while (left <= right) {  
        int mid = left + (right - left) / 2;  
  
        if (arr[mid] == key) {  
            result = mid;  
            left = mid + 1;  
        } else if (arr[mid] < key) {  
            left = mid + 1;  
        } else {  
            right = mid - 1;  
        }  
    }  
    return result;  
}
```



Count Occurrences

```
count = lastOccurrence - firstOccurrence + 1
```

Time: $O(\log n)$

Search in Rotated Sorted Array

Time: $O(\log n)$

```
static int searchRotated(int[] arr, int key) {  
    int left = 0, right = arr.length - 1;  
  
    while (left <= right) {  
        int mid = left + (right - left) / 2;  
  
        if (arr[mid] == key) return mid;  
  
        if (arr[left] <= arr[mid]) {  
            if (key >= arr[left] && key < arr[mid])  
                right = mid - 1;  
            else  
                left = mid + 1;  
        }  
    }  
}
```



```
    } else {  
        if (key > arr[mid] && key <= arr[right])  
            left = mid + 1;  
        else  
            right = mid - 1;  
    }  
}  
return -1;  
}
```

Search Insert Position

Time: $O(\log n)$

```
static int searchInsert(int[] arr, int key) {  
    int left = 0, right = arr.length - 1;  
  
    while (left <= right) {  
        int mid = left + (right - left) / 2;  
  
        if (arr[mid] == key) return mid;  
        if (arr[mid] < key) left = mid + 1;  
        else right = mid - 1;  
    }  
    return left;  
}
```



Exponential Search

Used when:

- Array size unknown
- Unbounded search space

Time: $O(\log n)$

THIRAVUGAL

Complexity Comparison

Algorithm	Time	Space
Linear Search	$O(n)$	$O(1)$
Binary Search	$O(\log n)$	$O(1)$
Recursive Binary	$O(\log n)$	$O(\log n)$



Common Mistakes

- Using binary search on unsorted data
- Wrong mid calculation
- Infinite loops
- Incorrect boundary update
- Forgetting base condition

Interview Discussion Points

- Why $O(\log n)$ is powerful?
- Why binary search needs sorted data?
- How to modify binary search?
- Difference between iterative and recursive?
- Why binary search beats linear at scale?



Performance Thought Exercise

Dataset size:

1 million elements

Linear search:

1,000,000 checks

Binary search:

~20 checks

Major scalability difference.

Summary

Searching algorithms provide:

- Fundamental optimization
- Logarithmic performance
- Core idea of divide & conquer
- Base for advanced algorithms

Master:

- Binary search template
- First/last occurrence
- Rotated array search
- Boundary conditions